Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Secure CBT: Hierarchical Key-Chained Question Paper Delivery & Leak-Traceability System

Team Name: Secure CBT *Team #: DSCPP-III-2026-T145* *Semester: 3rd Section: BTECH* *Mentor: Mr. Kartikey Arora*	
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Naman – 2510013438* *2510013438@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Gupta, Aayush – 2510370185* *2510370185@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Bhushan, Kushagra – 2510012263* *2510012263@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

- Secure CBT is a computer-based test question-paper system built around a single idea: a paper that leaks should still be traceable back to exactly where it leaked from.
- It targets the three points where a real exam paper is actually vulnerable: when the paper is set by teachers, when it is distributed down the exam hierarchy, and at the exam centre itself before the test begins.
- Rather than adding tracing as an afterthought (access logs, attendance records, duty rosters), the system builds traceability directly into the encryption: every node in the distribution hierarchy holds its own uniquely derived key, via hierarchical XOR key-chaining combined with AES-256, so a leaked ciphertext is its own fingerprint.
- Phase 1, the paper-setting pipeline (CSV upload, validation, question storage, paper generation, compression, and encryption), is built and running. Phase 2, secure distribution across the server hierarchy, is in progress. Phase 3, exam-centre decryption and leak tracing, is planned.

Updated Project Approach and Architecture (2 pts)

- The system is written in C++17 for all core data structures and algorithms, chosen for fine-grained control over memory and performance, which matters when every node in the hierarchy has to run its own encryption and lookup logic.
- OpenSSL provides the cryptographic primitives: AES-256 for encrypting the paper at each node, and SHA-256 for the tamper-evident transfer log, so the project relies on audited, standard implementations rather than hand-rolled cryptography.
- A web frontend handles the teacher-facing side: question-bank upload, CSV validation feedback, and template download, so the backend never has to deal with malformed input directly, and teachers get immediate feedback instead of a silent failure.
- Git/GitHub is used for version control, with all three members pushing to the shared repository, which also gives a clear history of who implemented which module.

Phase 1 (built) — Paper Setting

- Teachers upload a CSV question bank through the web screen, following the downloaded template, so no single teacher controls the final assembled paper and no manual compilation step is needed.
- The CSV checker reads the header and every row. It rejects the file outright if the header does not exactly match the template, and otherwise reports row-level errors (row number, field, and problem) so the teacher can fix and re-upload without guessing what went wrong.
- A file is accepted only once it contains at least one valid question; a header-only file is reported as empty rather than silently accepted as a zero-question paper.
- Valid questions are stored in a hash map (QuestionBank) for O(1) lookup by ID, and indexed by topic in a second hash map (TopicIndex), so the PaperGenerator can pull candidate questions per topic without scanning the whole bank.
- PaperGenerator selects N questions per topic using random selection without replacement, removing each chosen ID from the pool so it cannot repeat, which keeps the final paper balanced across topics rather than skewed toward whichever questions happen to be picked first.
- The selected questions are serialised into a single byte stream and compressed using Huffman coding: a frequency table is built from the byte stream, a min-heap merges nodes into a Huffman tree, and the resulting prefix-code table replaces common byte patterns with shorter bit codes.

- The compressed paper is then encrypted. Compression always runs before encryption, since encrypted data has no repeating patterns left for Huffman coding to exploit, so doing it the other way around would waste the compression step entirely.

Phase 2 (in progress) — Secure Distribution

- The distribution structure is an N-ary tree, implemented as a Left-Child Right-Sibling (LCRS) binary tree: each node stores a `firstChild` pointer and a `nextSibling` pointer, so any number of real children (states, districts, centres) can be represented with exactly two pointers per node instead of a variable-length child list.
- The paper is broadcast down the tree breadth-first (level by level), so every node at one level receives its copy before the next level begins, and transfers only ever move from parent to child, which rules out any accidental back-flow by construction.
- Each node's copy is encrypted independently using XOR key-chaining combined with AES-256: a node's key is derived from its parent's key XORed with the node's own secret, so decrypting at any level requires both pieces and no level in the chain can be skipped.
- Keys are derived on demand rather than pre-computed and stored in bulk, so only a small secret value is kept per node, which keeps memory overhead flat even as the hierarchy grows to cover many states and districts.
- An AVL tree indexes every node by ID, giving guaranteed O(log n) lookup of that node's secret, assigned key, and current status, even as nodes are added, which avoids the worst-case linear lookup a plain unbalanced tree could fall into.
- Every transfer between nodes is appended to a hash-chained log: each entry stores the SHA-256 hash of the previous entry plus its own data, so changing or removing any past entry breaks every hash after it in the chain, making tampering immediately detectable rather than something discovered later by chance.

Phase 3 (planned) — Exam Centre

- The exam centre server authenticates each student terminal and downloads the encrypted paper package addressed specifically to that centre, so a terminal cannot pull a paper meant for a different centre.
- Decryption is time-locked: the centre cannot read the paper before the scheduled exam start time, enforced by a time check, or by a min-heap Extract-Min when a single centre has to manage several exam slots and needs to know which one unlocks next.
- Decryption at the terminal uses the centre's key together with the node's own secret, following the same key-chain established in Phase 2, so Phase 3 reuses the existing key scheme instead of introducing a separate one.
- If an encrypted paper file ever surfaces publicly, it is matched directly against the AVL registry carried over from Phase 2 to identify the exact centre or terminal it was issued to, without needing any separate fingerprint database.

Tasks Completed (7 pts)

Task Completed	Team Member
Key manager (XOR key derivation, encrypt/decrypt)	Naman
QuestionBank, Teacher, and PaperGenerator classes	Naman
Question.h and QuestionBank.h	Naman
TopicIndex.h and PaperGenerator.h	Aayush
Question-bank upload screen (frontend)	Kushagra
Huffman compression and encryption	Kushagra
CSV checker (in progress)	Kushagra

Challenges/Roadblocks (7 pts)

Formatting (solved)

- CSV files from Excel broke the parser, mainly with commas inside question text and UTF-8 byte-order marks.
- Fixed by requiring the template format, reading quoted fields correctly, and reporting row-level errors instead of failing silently.

Phase 2 transfer (expected)

- Moving the encrypted paper between nodes is the next risk.
- Plan: simulate transfers in-process with function calls first, verify the tree broadcast and key chain, then add a network layer later.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
LCRS tree and BFS broadcast	Naman
AVL node registry	Aayush and Naman
Presentation (PPT)	Aayush
XOR key-chaining with AES-256 per node	Kushagra
SHA-256 hash-chained log	Kushagra
Documentation and report	Kushagra
Transfer layer between nodes (simulated first)	Kushagra
Time-locked decryption at the centre	Naman
End-to-end simulation and leak-tracing evaluation	Aayush

Project Outcome/Deliverables (2 pts)

Phase 1

- A working paper-setting pipeline that takes a teacher's CSV upload all the way through to an encrypted, compressed paper, with no manual steps in between.
- CSV validation that catches a bad header or a bad row before it ever reaches the paper-generation logic, reporting the exact row, field, and problem back to the teacher.
- Complete, documented C++ source for QuestionBank, TopicIndex, PaperGenerator, the Huffman compressor, and the key manager.
- A question-bank upload screen with all three states (uploading, success, and error) wired to real backend output, not placeholder text.

Phase 2 (planned)

- A working LCRS tree that broadcasts the encrypted paper down the server hierarchy using breadth-first traversal.
- Per-node encryption via XOR key-chaining and AES-256, so every node's ciphertext is uniquely keyed by its position in the hierarchy.
- An AVL-based node registry giving O(log n) lookup of any node's secret, key, and status.
- A SHA-256 hash-chained transfer log, with a demonstration that tampering with any past entry is caught immediately.

Phase 3 (planned)

- Time-locked decryption at the exam centre, so no readable paper exists before the scheduled exam time.
- A multi-centre simulation covering several states, districts, and centres end to end.
- A leak-tracing demonstration that takes a simulated leaked ciphertext and identifies its source node using only the AVL registry.

Progress Overview (2 pts)

- Phase 1 is functionally complete: the full paper-setting pipeline (CSV upload, validation, QuestionBank/TopicIndex, PaperGenerator, Huffman compression, and encryption) is built and committed to the repository, along with the matching frontend upload screens.
- The CSV checker code is complete; formal test logging (header-only, valid file, and row-error cases) is the one remaining item before Phase 1 can be marked fully validated.
- Phase 2 has started ahead of the original plan: initial design and groundwork for the LCRS tree are underway, though the AVL registry and the hash-chained log have not yet been implemented.
- Nothing is currently behind schedule. The main risk going forward is the node-to-node transfer mechanism in Phase 2, which is why it is being simulated in-process first before any real networking is added.

Codebase Information (2 pts)

- **Repository:** `https://github.com/Kushagratriest/Secure-CBT`
- **Branch:** main
- **Latest update:** Question handling and project configuration, pushed by Naman
- **Key files added:** Key manager (encryption keys), question bank and question storage, topic index, paper generator

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
CSV checker, header-only file	Pass	Expected: Question bank can be uploaded.
CSV checker, valid 2-row file	Pass	Expected: count 2
CSV checker, row errors	Pass	Expected: row, field, and problem reported
Phase 1 end-to-end pipeline	In progress	CSV to encrypted, compressed paper
Huffman round-trip	In progress	Decompresses to original text
Phase 2 transfer and broadcast	Not yet run	Planned after LCRS is built

Deliverables Progress (2 pts)

- Phase 1: backend built, CSV checker in progress, upload screen built.
- Phase 2: in progress, with LCRS and AVL next.
- Phase 3: planned.